

LightVA: Lightweight Visual Analytics With LLM Agent-Based Task Planning and Execution

Yuheng Zhao¹, Junjie Wang, Linbing Xiang², Xiaowen Zhang, Zifei Guo³, Cagatay Turkey⁴, Yu Zhang⁵,
and Siming Chen⁶

Abstract—Visual analytics (VA) requires analysts to iteratively propose analysis tasks based on observations and execute tasks by creating visualizations and interactive exploration to gain insights. This process demands skills in programming, data processing, and visualization tools, highlighting the need for a more intelligent, streamlined VA approach. Large language models (LLMs) have recently been developed as agents to handle various tasks with dynamic planning and tool-using capabilities, offering the potential to enhance the efficiency and versatility of VA. We propose LightVA, a lightweight VA framework that supports task decomposition, data analysis, and interactive exploration through human-agent collaboration. Our method is designed to help users progressively translate high-level analytical goals into low-level tasks, producing visualizations and deriving insights. Specifically, we introduce an LLM agent-based task planning and execution strategy, employing a recursive process involving a planner, executor, and controller. The planner is responsible for recommending and decomposing tasks, the executor handles task execution, including data analysis, visualization generation and multi-view composition, and the controller coordinates the interaction between the planner and executor. Building on the framework, we develop a system with a hybrid user interface that includes a task flow diagram for monitoring and managing the task planning process, a visualization panel for interactive data exploration, and a chat view for guiding the model through natural language instructions. We examine the effectiveness of our method through a usage scenario and an expert study.

Index Terms—Visual analytics, task planning, large language model agent, mixed-initiative interaction.

I. INTRODUCTION

VISUAL analytics (VA) deciphers complex datasets with data mining and interactive visualizations [1], [2]. However, building and using a VA system can be a costly endeavor that encompasses several main stages: goal understanding, task decomposition, data modeling, and visualization creation to

discover insights. A key challenge is that this process is iterative, requiring continual refinement based on evolving needs [3]. Different tasks necessitate various data analysis and visualization methods to form a VA system. While using the system, tasks may evolve based on the insights gained, necessitating ongoing iterations until the analytical goals are achieved [4]. Consider a scenario where the goal is to identify high-risk events from social media data. Users may first need an overview from different perspectives, such as the distribution of keywords over time or changes in sentiment for risk analysis. If outliers are identified, users may need further details, such as spatial distribution or entity relationships. In this process, the task space is broad and fluid, requiring efficient task planning and method implementation.

Recent research focuses on data-driven or natural language-based visual data exploration, with an emphasis on automatic visualization generation [5], [6] or insight mining [7], [8]. Large Language Models (LLMs) present a potential for data analysis, supporting dynamic task planning and lower development costs across various scenarios. The reasoning abilities that enable autonomous planning and execution of analytical tasks [9], [10], [11], while code generation capability support creating insightful visualizations efficiently [12], [13], [14]. Additionally, their broad knowledge base makes LLMs versatile tools capable of adapting to diverse data analysis contexts [15], [16]. LEVA [17] integrates LLMs in VA systems to recommend insights for a given task but still cannot generate visualization and data modeling methods adapted to tasks. There is a lack of approaches supporting task planning, VA methods implementation, and interactive analysis with human-agent collaboration.

This paper introduces *LightVA*, a lightweight VA framework with agent-based task planning. The term “lightweight” refers to the framework’s focus on reducing the cost of development and using VA systems. Using LLM agents to aid the task planning and execution process. The framework builds upon multi-level relationships, translating high-level goals to low-level tasks and deriving insights through data mining and interactive visualizations. Specifically, the framework employs a recursive process that includes a planner, executor, and controller, which dynamically accommodates task complexity. The planner is responsible for task decomposition, the executor handles task execution, including visualization generation and data analysis, and the controller orchestrates the executor and planner and manages whether tasks continue to be decomposed. We develop a system based on the framework that provides a chat view to support

Received 26 July 2024; accepted 27 October 2024. Date of publication 14 November 2024; date of current version 1 August 2025. This work was supported by the Natural Science Foundation of China NSFC under Grant 62472099. Recommended for acceptance by A. Lex. (Corresponding author: Siming Chen.)

Yuheng Zhao, Junjie Wang, Linbing Xiang, Xiaowen Zhang, Zifei Guo, and Siming Chen are with the School of Data Science, Fudan University, Shanghai 200437, China (e-mail: yuhengzhao@fudan.edu.cn; simingchen@fudan.edu.cn).

Cagatay Turkey is with the Centre for Interdisciplinary Methodologies, University of Warwick, CV4 7AL Coventry, U.K. (e-mail: Cagatay.Turkey@warwick.ac.uk).

Yu Zhang is with the Department of Computer Science, University of Oxford, OX1 2JD Oxford, U.K. (e-mail: yuzhang94@outlook.com).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TVCG.2024.3496112>, provided by the authors.

Digital Object Identifier 10.1109/TVCG.2024.3496112

communication among users and agents, a task flow view to visualize manage the process of task planning, a visualization panel to show single view and multiple linked views connected to the task flow. Our main contributions are as follows:

- We propose a lightweight VA framework using LLM agent-based task planning and execution. This approach enables adaptive, efficient analysis through human-agent collaboration, supporting users in task decomposition, visualization, and insight discovery.
- We develop a system that embodies our framework, supporting users to analyze data with the assistance of agents and communicate through the hybrid user interface.
- We demonstrate the effectiveness of the system through a usage scenario and an expert study.

II. RELATED WORK

Our research is related to prior studies on visualization recommendations, task-driven data exploration, and LLM application in data exploration.

A. Visualization Recommendation

Visualization authoring typically requires users to have professional visualization knowledge and programming ability. For example, tools like Tableau support creating visualizations and multiple linked views with shelf-configuration design, offering robust visualization creation features. However, while Tableau excels as an authoring tool, it offers limited support for automatic task decomposition and VA method implementation to further assist with analysis. A corpus of research has been proposed on automatic visualization recommendations [18], [19], [20]. Rule-based methods, such as Voyager [21] and CompassQL [22], utilize the visualization principles to construct visual mapping and allow users to choose their interested data properties and visual encoding to create visualizations. For machine learning methods, Data2Vis [23] introduces an end-to-end trainable neural translation model for automatically generating visualizations from given datasets. VizML [24] learned visualization design choices from a corpus of data-visualization pairs. Table2Charts [25] recommends visualizations by learning patterns between tables and visualizations. ChartSeer [26] employs deep learning to recommend visualizations based on users' interactions. Unlike end-to-end deep learning methods that directly learn from datasets to generate visualizations, knowledge graph-based approaches, such as AdaVis [27], KG4VIS [28], Lodestar [29], leverage structured information about data and relationships to recommend visualizations.

In addition, there are some works that consider multi-view generation. Qu and Hullman [30] proposes coordination principles to keep consistency. Sun et al. [31] investigate different linking techniques based on data relationships. Dziban [32] is a visualization API using anchored recommendation and extending Draco [33] to reason about multiple views. DMiner [34] investigated the design rules of the single views and view-wise relationships from online notebooks to recommend multiple-view dashboards. MultiVision [35] and DashBot [5] recommend dashboards given an input dataset in an end-to-end manner

using deep learning models. Shi et al. [36] optimize multi-view layouts by predicting the similarity of visual elements using Transformer-based models. Previous work has provided a solid research foundation for the principles between data, visualization, and multi-views. Building on this, we further study integrating LLM-agent to recommend visualizations that align with high-level goals and evolving tasks throughout the VA pipeline, which require significant human effort.

B. Task-Driven Visual Data Exploration

In addition to data attributes when recommending visualizations, some visualization recommendation systems are task-driven recommendation systems that consider one or more analytic tasks (e.g., correlate, analyze trend). Some scholars study the recommendation of analysis methods in exploratory data analysis (EDA) within notebooks. EDAssistant [37] recommends code by analyzing associations between APIs in a large notebook collection. ATENA [38] shapes EDA into a Markov Decision Process (MDP) model using a deep reinforcement learning architecture to effectively optimize notebook generation. Furthermore, visual analytics incorporates visualization techniques into the EDA process, expanding the task space. Casner [39] presents one of the earliest examples of visualization systems that suggest charts based on a user's task (e.g., finding direct flight routes or a table to see flight information). Saket et al. [40] conducted a study to assess the effectiveness of five canonical visualizations on ten low-level analytic tasks [41] and developed a recommendation engine based on their study's findings. Gotz and Wen [42] present a prototype system that observes interaction patterns (e.g., repeatedly changing filters or swapping attributes) to infer analytic tasks such as comparison or trend analysis and correspondingly recommends visualizations such as small multiples or line charts. VizAssist [43] enables its users to specify their data objectives in terms of analytic tasks (e.g., correlate, compare) and considers these tasks in combination with existing perceptual guidelines as input to a genetic algorithm for recommending visualizations. Foresight [8] uses tasks like distributions, outliers, and correlations to guide insight discovery and grouping recommendations. TaskVis [44] recommends visualizations under specific tasks through answer set programming.

In addition to visual generation based on a single task, Medley [45] recommends multi-view collections based on several analytic intents, and views and widgets can be selected to compose a variety of dashboards. However, the analytic intents and visualization combinations are often chosen from preset options, which limits exploration flexibility. Different from them, we study dynamic task planning based on the goal and findings and leverage human-agent collaboration.

C. Large Language Model in Data Exploration

LLM-based tools have been proposed for data exploration and visualization tasks. For visualization generation and recommendations, LLM4Vis [14] and ChartGPT [46] utilize LLMs to choose appropriate visualizations from natural language instructions. Li et al. [47] evaluate the capability of GPT-3.5

to generate visualization specifications, demonstrating its superiority over previous machine learning-based approaches. NL2Rigel [48] showcases the LLM's ability to convert instructions into comprehensive data visualizations and tables. For analytical task translation and automation, Hassan et al. [49] and Data-Copilot[16] concentrate on converting analytical goals and ambiguous queries into actionable data analysis tasks. Ma et al. [50] and JarviX[12] introduce systems that automate the data exploration process by identifying suitable analysis intents and generating insights. Text2Analysis [51] offers a framework for categorizing data analysis tasks, establishing a structured approach to tackling common analytical challenges ranging from basic operations to forecasting and chart generation. However, the tasks in these studies are generally straightforward and focused, with limited exploration into the decomposition of more complex tasks.

When using LLMs to solve complex tasks where multi-step reasoning is demanded, the performance of directly using LLMs tends to decrease. Recently, prior methods have utilized LLMs with input-output prompting, CoT [52], ToT [53] or GoT [54] to perform complex task planning and execution. These methods proved that LLMs are good at task planning but require appropriate prompting techniques. Another way is to integrate LLMs in the interface, allowing chaining multiple prompts to address a much wider range of human tasks. Wu et al. [55] introduce the chaining of AI models, where a complex task is divided into multiple steps. Talk2Data [56] presents a natural language interface that enables users to explore visual data through question decomposition. However, the linked visualization and more advanced data analysis methods remain limited. To enhance this capability, we leverage LLMs and propose an agent-based autonomous task-planning strategy for adaptive VA system construction and exploration.

III. LIGHTVA FRAMEWORK

The pipeline of VA involves two stages: development and analysis. Thus, in LightVA, we aim to reduce the efforts for both developers and analysts. In the following, we will examine the challenges and derive the design requirements for integrating LLM-based agents into the user's workflow. Finally, we introduce the conceptual framework of agent-based VA workflow.

A. Challenges

Through a review of visual analytics literature, we identify and dissect specific challenges that intensify the effort users must exert:

- C1 *Accommodating diverse analysis requirements:* Data Analysts often face the immense challenge of navigating a vast exploration space, where the analytical process is dynamic and iterative [4]. Forming hypotheses and validating them through the continuous proposal of new tasks and insights is complex. They have to figure out the connections between tasks and insights in their mind and try to propose tasks in the next few steps until they achieve the goal.

- C2 *Developing visualization and data mining tools is time-consuming:* Analysts often lack proficiency in developing and managing VA systems, which makes it challenging for them to select and apply appropriate data modeling and visualization techniques [45]. Additionally, when tasks involve multiple visualizations, these must be integrated into a linked view to enable more effective interactive exploration [57]. However, this process requires substantial knowledge of both visualization principles and coding skills, resulting in inefficiencies and delays [58].

B. Design Requirements

Based on these challenges, we have settled on a set of targeted design requirements.

- R1 *Adaptive task planning:* Task proposals need to be tailored to users' analysis goals and the given dataset. This includes exploring in depth and breadth, with automatic planning and execution reducing user efforts. Moreover, as exploration results emerge, new tasks should contextually link to previous exploration results, adapting to the switching between tasks. (C1)
- R2 *Flexible visualization generation:* The generation of visualizations should flexibly handle different tasks and data. This encompasses accurate data identification, transformation, and selection of visualization types, as well as adding highlights based on discovered insights to reduce cognitive load. Furthermore, generated views should support interaction, allowing for more immersive user analysis. (C2)
- R3 *Automatic insight generation:* To reduce development costs, the system should efficiently complete code-based data analyses for given tasks, providing visualizations and suggesting findings to facilitate hypothesis forming and validation. To lessen the cognitive burden, important parts of insights should be highlighted in rich text format in the output results. (C2)
- R4 *Multiple view composition:* As new visualizations are added, older ones may become less relevant. To display the most recent results to the user, the visualization panel needs to be updated. However, it is crucial to avoid discarding previous results as they may be relevant to new insights. Users should be allowed to merge visualizations of interest even if they are not the latest. Within the merged views, users can focus their analysis on a smaller scope through interaction, reducing cognitive load among large sets. (C2)
- R5 *Intuitive analysis process:* The relationship between tasks, visualization, insights, and analysis progress should be presented in a more intuitive form. The system should support interactions between human and agent to help users understand the agent's task planning and execution. (C1, C2)

C. Conceptual Framework

Based on the challenges and requirements discussed, we propose LightVA, a lightweight VA framework with agent-based

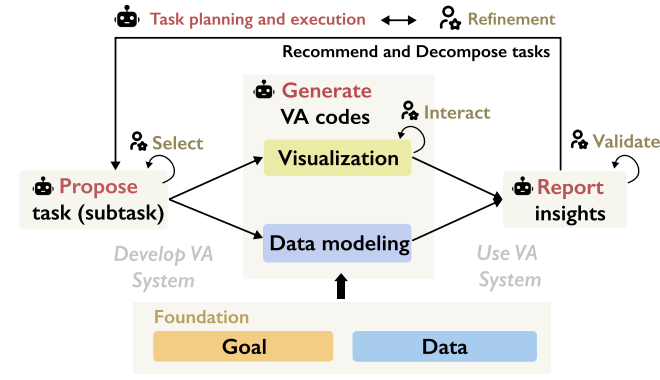


Fig. 1. The illustration of conceptual framework in LightVA. The agent creates the VA system for analysis by task planning and execution, while the user uses the created VA system and refines agent outputs.

task planning. The “lightweight” refers to the light expectations in developing VA systems and using the developed systems for analysis. We design the framework to involve a recursive task-solving process in which goals and data are inputs, and insights are outputs. The intermediate results are tasks, subtasks, visualization, data modeling (Fig. 1). The LLM agents are integrated to support goal understanding and task decomposition (R1), data modeling and visualization codes generation (R2, R3), and linked view generation for interactive exploration (R4). Meanwhile, the users can monitor the process, guide the agent, and refine the agent’s output through direct manipulations and natural languages (R5).

1) *Defining Primitives*: According to the above descriptions of the framework, some key primitives need to be defined.

Goal and Data: We define a goal as an overall description or a vague utterance that expresses the high-level purpose of the analysis. For instance, “to analyze the influence factor on a car’s fuel efficiency”. The goal directs the focus of the analysis, and the data supplies the analysis evidence to meet the goal.

Task: Tasks are specific aspects derived from the goal and make the goal executable. We define them using four attributes

$$\text{task} := \langle \text{type}, \text{data variables}, \text{method}, \text{progress} \rangle. \quad (1)$$

The *type* indicates the nature or category of the analytical operation to be performed, such as finding extreme, outlier, change point, trend analysis, etc. The *data variables* are the objects upon which the tasks will be applied. The *method* defines how the task will be solved, including data modeling and visualization methods, and the solved result is *insight*. If the task is complex, which means it covers multiple *data variables* and needs to use multiple *methods* to solve, the task can be decomposed into *subtasks*. To depict whether the task is completed, we define *progress*, which means the task is solved when its subtasks are solved as well. Given that the tasks can be complex and range from low to high abstraction, we describe them using natural language.

Insight: This denotes the expected or targeted knowledge or understanding that the task aims to achieve. We define insight using four attributes

$$\text{insight}$$

$$:= \langle \text{type}, \text{parameters}, \text{data variables}, \text{data values} \rangle. \quad (2)$$

The *insight type* can be discoveries, patterns, trends, or anomalies identified through analysis [59]. For example, if the task type is “compare”, the insight type could be “difference” [7]. The *parameters* specific features of the insight, such as “increasing” or “decreasing” of “trend”. The *data variables* for insights can be the original data columns or transformed variables, while the *data variables* for tasks are the original data columns. For example, if the task involves analyzing vehicle weight and fuel efficiency (MPG), the task’s *data variables* would include “Weight_in_lbs” and “MPG”. If the insight includes a derived metric such as “MPG per pound,” this would be considered a transformed variable specific to the insight. The *data values* refer to particular values of *data variables*. For example, the maximum MPG value is “48”.

Visualization: Refers to the visual representation required for the task that best conveys the data and insights.

$$\text{visualization}$$

$$:= \langle \text{type}, \text{encoding}, \text{interaction}, \text{coordination} \rangle. \quad (3)$$

Our framework generates visualizations using Vega-Lite grammar [60], which supports the above four aspects. The types of *interactions* are, e.g., filtering, zooming, and hovering supported by the visualization. The *coordination* describes how this visualization interacts or synchronizes with other visualizations, such as a brush, to filter each other.

2) *Workflow of Framework*: Our framework involves building a *task flow*, represented as a directed compound graph $G = (V, E)$, where each *node* $v \in V$ represents a *task* and *edge* $e \in E$ represents the connection from one task to another task. Since the process of exploration involves both enlightening and in-depth thinking, we define two types of task edges: *recommendation* and *decomposition*. And we define the solving of the task as *execution*. The difference between recommendation and decomposition is that recommendation is “goal-oriented”, and decomposition is “task-specific”. The recommendation aims to broaden the exploration scope, providing heuristic suggestions. The decomposition aims to ensure a detailed plan with clear logic for solving the task. The following sections will introduce how humans and agents collaborated to complete the analysis in our framework (Fig. 2).

Stage1. Task recommendation: Initially, the user uploads the data and inputs a goal (H1), and then the agent interprets data (A1) and transforms the goal into specific, actionable tasks (A2). The user can provide feedback and accept or modify recommended tasks to align with their analysis needs (H2). Specifically, the interaction process involves two stages:

- *Initial stage*: At the beginning, agent should first interpret the data and propose tasks that make the goal executable by mapping it to the data. The principle is to identify *data variables* and *task type*. For instance, for a goal of “find high-risk events in a city”, useful data variables could include time, space, text, and sentiment. The agent also needs to distinguish the purpose of the analysis, e.g., “finding outliers”. Thus, different combinations of data

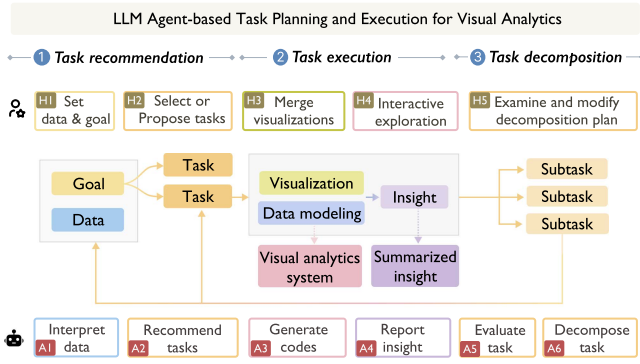


Fig. 2. The workflow of the LLM agent-based task planning and execution for visual analytics. The collaboration between users and the AI agent is characterized by three stages: task recommendation, task execution, and task decomposition. The user proposes goals, selects tasks, merges visualizations, and engages in interactive exploration (H1–H5). The agent interprets data, recommends tasks, generates codes, reports insights, evaluates tasks, and decomposes tasks (A1–A6).

variables and task types can form different tasks under a goal.

- *Historical context stage*: As the analysis continues to accumulate in the recommendation process, the agent should recommend tasks considering previous tasks and the overall goal. First, when a task is completed, the agent evaluates if the overall goal has been achieved. If not, it should recommend new tasks aligned with the goal. Second, previously unexplored tasks should be re-evaluated and proposed again if they are still relevant to the goal.

Stage2. Task execution: In this stage, the agent generates visualization and modeling codes (A3) and executes the codes to report and summarize insights (A4). Having multiple visualizations and modeling approaches, users could select visualizations to merge a linked view (H3) and interactively explore it (H4). It comprises the following two-step approach:

- *Visualization and insight generation*: When the user selects an agent-proposed task or proposes a task themselves, the agent writes codes to complete the analysis. The agent needs to choose appropriate data modeling and visualization *methods* for each *task* and run the codes to complete the analysis. The agent then needs to generate the *insights* into a structured format. Finally, the agent should summarize insights obtained from the decomposition of subtasks, providing a comprehensive overview of the analysis.
- *Multi-view linking*: Users can initiate *coordination* by selecting multiple visualizations they prefer while the agent generates codes. We currently do not automatically combine visualizations from subtasks because subtasks decomposed from a main task often share common variables and visualization types. For example, several subtasks might use latitude and longitude to create maps exploring different spatial patterns. The linked view is often used to conduct multi-variate association analysis with different visualizations. Thus, allowing users to choose their visualizations can provide a more flexible analysis.

Stage3. Task decomposition: We design the decomposition following a “on-demand” strategy. In other words, execution is prioritized, and decomposition is considered only if the task is not completed. This approach aims to ensure users can see initial results quickly in an interactive environment. In this stage, the agent is responsible for evaluating tasks (A5) and proposing a decomposition plan (A6), while the users can examine and modify agent output to override the agents (H5).

- *Results assessment*: Based on the initial execution’s insights and the complexity of the task, the agent assesses the need for further analysis. According to the definition of *task*, the agent should verify the selection of *data variables* and the rationality of data modeling and visualization *methods*. The evaluation should be explained to make users understand the motivation of decomposition.
- *Sub-task generation*: If decomposition is necessary, agent should formulate a plan outlining subtasks. Each subtask should have appropriate *data variables* and *methods* addressing distinct aspects of the main task. These subtasks should have an execution order, e.g., in parallel or sequentially.

IV. LIGHTVA SYSTEM

Guided by the design requirements, we propose a pipeline of LLM agent-based task planning and an interface to support interactive visual data exploration with assistance.

A. Agent-Based Task Planning

Based on the conceptual framework previously introduced, we propose an agent-based task-planning pipeline Fig. 3. In this process, we have three kinds of modules. The *planner* has two types: *recommender* and *decomposer*, which are responsible for task recommendation and decomposition respectively. The *executor* handles task execution, including visualization generation and data analysis, and the *controller* is the recursive algorithm that bridges the *executor* and *planner* (See the Algorithm 1 in the appendix), available online. When introducing each stage’s strategy, we provide the prompt templates that describe input and output, instructions to LLMs, and indicators to describe the output format. The prompt examples and outputs are available in Appendix A, available online.

1) Task Recommendation: Task planning begins with a recommendation approach based on the given goal and dataset. The *recommender* aims to transform the goal into actionable tasks. The implementation follows the step-by-step guidelines with two stages: initial stage and historical context stage (Prompt template 1).

In the initial stage, the input is the goal and the data (Fig. 3(a)) and the objective is to convert the goal into actionable tasks. Based on the framework (Section III-C2), we guide the model with three principles. First, identify *data variables* relevant to the *goal*. Second, propose several *task type* to refine the goal from different aspects. Third, draft descriptions of tasks by combining these data variables and task types.

In the historical context stage (Fig. 3(b)), the generated tasks should be divided into two types: linking goals with discoveries

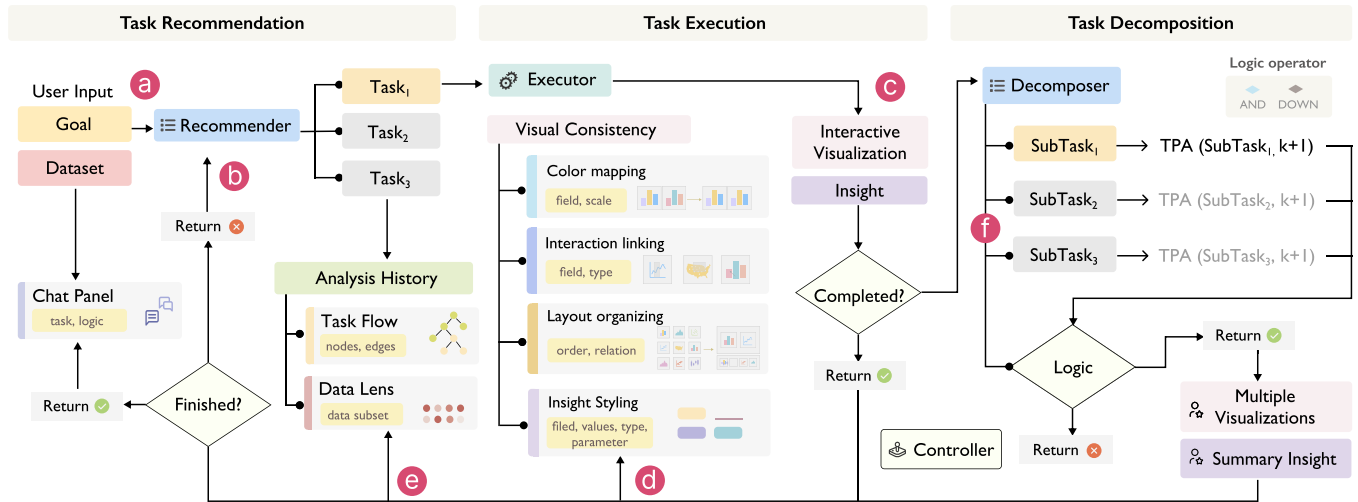


Fig. 3. The agent-based system architecture. The pipeline starts from the goal and data with an agent-based task planning strategy, including recommendation, execution, and decomposition. Specific components and interactions are labeled (a)–(f), where (a) is the initial stage to recommend tasks from the goal, (b) recommends tasks with historical contexts, and (c) is the execution of tasks generating visualization and insights. After that, a series of optimizations for visual consistency are performed (d). Historical records are managed, and the progress of tasks is updated in a timely manner (e). According to the analysis results, the agent evaluates whether the task needs to be decomposed (f).

Prompt Template 1 (Task Recommendation).

—Initial stage

Input: {goal, data}

Instruction: You need to come up with a short plan based on the understanding of the data to help accomplish the goal. Please recommend n exploratory tasks, including task description, type, and data variables. For task type, you may consider the trend, correlation, category, distribution, etc, to explore the goal from different aspects. For data_variables, list the original column names.

Indicator: {An example in JSON format}

Output: {new tasks}

—Historical context

Input: {goal, data, explored and unexplored tasks}

Instruction: You need to supplement some new tasks for explored tasks by considering the results of tasks already explored if needed. Second, it is recommended that previous unexplored tasks be revisited if they are suitable for analysis at this stage by considering the explored tasks.

Indicator: {An example in JSON format}

Output: {new tasks, existing tasks to review}

to propose *new* tasks and tasks that might have been forgotten by the user needs to *review*. To achieve this goal, the model might need to consider the following steps. First, evaluate completed tasks to check if the overall goal is achieved. Second, propose new tasks based on the current context and previous tasks. Third, propose previously unexplored tasks if they remain relevant. In addition to guidelines, we should provide examples of output to guide models.

2) *Task Execution*: After tasks are proposed and confirmed by the user, the executor will solve tasks by generating codes (Fig. 3(c)). We then adopt the “decompose on-demand” strategy, which means we first execute the task and then decompose the task if the results are unsatisfactory. This approach allows

users to obtain preliminary results quickly and enables a more flexible way to address both simple and complex tasks, avoiding unnecessary time cost that comes from always decomposing tasks in advance.

To execute a task, the input for the *executor* includes a goal, data, and task description, and the output includes visualizations and insights for the selected task (Prompt template 2). In the first round, two code snippets are generated: one for data analysis and the other for visualization. In the second round, structured insights are generated by data analysis results.

Visualization generation: For each task, the model needs to implement data analysis and visualization methods, where one task corresponds to one visualization, and the visualizations will be combined into multiple views when multi-variables association analysis is required, which we will introduce later. We use Vega-Lite via Altair [60] to generate visualizations. Vega-Lite is a high-level grammar that can support the generation of a variety of visualization types in a low-code way, and its declarative structure allows users to modify visualizations easily. As Altair only supports basic data transformation, we leverage Python’s flexible libraries, allowing for more complex analysis, such as regression and clustering. However, this may introduce inconsistencies and errors between analysis and visualization code snippets. To address this, we provide a code scaffold in the prompt to let LLMs fill the empty to enhance consistency and improve output stability.

Structured insight generation: After generating visualization and modeling codes, the executor needs to execute them to structure the results into insights. As the agent’s coding environment¹ does not support Altair, we use the local environment to execute the visualization part of the code. The agent runs the

¹<https://platform.openai.com/docs/assistants/tools/code-interpreter>, Last accessed March 2024. Code Interpreter allows Assistants to write and run Python code in a sandboxed execution environment.

Prompt Template 2 (Task Execution).*Code generation***Input:** {task, data summary, code template}**Instruction:** You need to write Python codes to analyze the data to solve this task. After finishing the data analysis, you should continue to use Altair to generate an interactive visualization. Add a brush or click function, a tooltip, and a legend if different colors are used.**Indicator:** {A code template}

```

1 import altair as alt
2 import pandas as pd
3 def plot(data: pd.DataFrame):
4     # Data preprocessing
5     <codes>
6     # Chart generation
7     chart = alt.Chart().mark_bar().encode()
8     return chart

```

Output: {insight, visualization}*Structured insight generation***Input:** {task, data, codes}**Instruction:** Run the codes to report an important insight for this task: task. You should output insight, including text, insight type, parameters, data variables, and data values. {Definitions of insight attributes.}**Indicator:** {A few examples in JSON format}**Output:** {insight}

data analysis part, interprets the results, and derives insights. We provide a format example according to our definition of insight in the framework to guide model output. This includes the insight description, the insight type, and the data variables and values. These attributes highlight key elements in the insight description to improve readability.

Linked-view generation: For multi-variable association analysis, we enable the generation of multiple linked views (Fig. 3(d)). While LLMs can add interactivity and set basic colors and layouts within the code, maintaining visual consistency across views, such as avoiding duplicate colors and ensuring neat layouts, requires implementing additional constraints.

- *Interaction linking:* To enable interaction linking among charts, we instruct the model to modify and combine the chart codes following several guidelines. First, ensure both charts contain common key columns with consistent data formats for the selection fields. Next, define the selection mechanisms. Common interaction methods include a brush on the time axis for the line chart, dual brushes for the scatter plot, and click interactions for the bar chart. Then, the transform filter should be applied to the other charts.
- *Layout organization:* When generating new visualizations, we set the charts to the same size and arrange them sequentially. If the user selects charts to merge multiple views, we allow selecting up to six charts to avoid excessive cognitive load. The LLMs should output the layout no more than three charts in a row.
- *Color mapping:* We built a data-to-color mapping rule within the task space based on Qu and Hullman's guidelines [30], where one color corresponds to one data dimension without reuse. For example, the same field should use the

same quantitative color scale across different views, while different fields should use non-overlapping hues or palettes to avoid confusion.

Task progress calculation: In the analysis process, each node has a *progress* attribute that quantifies the task completion degree. When the task is executed, the agent evaluates the task's completion status. If the task remains incomplete and needs further in-depth analysis, the progress is set to 0%; otherwise, it is set to 100%. Upon determining a task as incomplete, the agent suggests a decomposition plan. If the further decomposition is denied by the user, the progress is set to 100%. Each recently executed task is represented as a leaf node, initiating a bottom-up refresh of progress values for non-leaf nodes across the tree (Fig. 3(e)). The process for the main task is the average progress of two child tasks, which are user-confirmed nodes generated by task decomposition or recommendation. This hierarchical update process accurately reflects the analysis status and ensures progress values account for both the agent's assessments and user input.

3) *Task Decomposition:* Upon the execution of a task, the *decomposer* evaluates the quality of completion and decides whether to decompose and how (Prompt template 3). To evaluate if the task is completed, the input includes tasks, codes, and insight, while the output is a score ranging from 0 to 10 and an explanation. Then, if decomposition is needed (i.e., its score is below a certain threshold, like 8), a detailed decomposition plan with the appropriate logic operators will be proposed (Fig. 3(f)). Conversely, if no further decomposition is required, the *controller* calls the *recommender* to propose new exploratory tasks. According to the framework, the evaluation process can be guided by the following guidelines. The first is to determine if the initial solution adequately segments the *data*, ensuring that the analysis covers all relevant subsets. Second, assess if the task requires further data mining or visualization *methods* (e.g., regression, clustering) to extract deeper insights.

If a task requires decomposition, a depth-first decomposition process is employed. This process is guided by two factors. One is a predefined maximum decomposition step depth (k_{max}), managing the depth (k) of decomposition (decompose when $k < k_{max}$). Another factor is the completeness of subtasks. Here, we define each task that can be decomposed into subtasks with two types of logic.

- *AND:* Subtasks are independent and executed in parallel order with multiple agents.
- *DOWN:* This is a particular *AND* case. Subtasks are dependent and will be executed in serial order with a single agent.

We use an example to illustrate the decomposition process. If the task of *analyzing vehicle weight and fuel efficiency* is completed with a score of 6/10, the agent might suggest the following decompositions: T1: *Segment the data into weight categories (light, medium, heavy) and analyze the fuel efficiency within each segment.* T2: *Within each weight category, conduct a clustering analysis to identify patterns or groupings that could further explain variations in fuel efficiency.* T3: *Perform a multiple regression analysis to control for additional variables*

Prompt Template 3 (Task Decomposition).*—Is complete***Input:** {task, codes, insight}**Instruction:** You need to judge whether the task requires further analysis. The complexity of the task can be considered from data, data mining, and visualization methods. If the task appears incomplete, it needs to be decomposed further. Please rate the task from 1-10. For example, if the initial solution adequately segments the data or if the task requires advanced statistical analysis. You should output a score and explanation for your evaluation.**Indicator:** {A output template in JSON format}**Output:** {score, explanation}*—Decompose***Input:** {task, insight, score, explanation}**Instruction:** This task requires further analysis. According to the score and explanation, please generate no more than n subtasks and indicate the methods they use respectively based on this. There are two operators AND and DOWN in the execution order to connect tasks.**Indicator:** {An example in JSON format}**Output:** {subtasks, execution order}

like engine size and vehicle age. The execution logic would be (T1 DOWN T2 AND T3).

According to the logic, the agent executes each subtask and summarizes the subtasks' insights to formulate an overall insight for the decomposed task. No further decomposition is required if subtasks are all completed or up to the max steps. Then, considering the goal and analysis results, the agent might propose revisiting previously proposed tasks or recommending new exploration tasks.

B. LightVA Interface

The agent-based interface includes several views to enable user-controlled visual exploration, as shown in Fig. 4. To enable this, we provide four views: *Chat view*, *Visualization view*, *Task flow view*, *Data view*. The design of the interaction follows the design considerations of Section III-B and provides three modes of interaction.

Chat view: The exploration begins with the *Chat view* (Fig. 4(A)), an LLM-based chat box that facilitates direct communication between users and agents through natural language interaction. The interaction between the user and agent can switch between delegate, guide, and discuss. In this interface, the user uploads data and inputs goals. The agent then responds with its understanding of the dataset and suggests new tasks in the form of buttons. Users can bookmark tasks of interest, and tasks that are not of interest will not be counted in the progress of exploration. The user can then execute a task by clicking on it. Additionally, users can enter their own tasks in the chat box. After submitting a task, the agent will provide a visualization of the production and insights obtained from the calculations in the *Visualization view* (Fig. 4(B)). If the task needs to be decomposed, the task decomposition plan will appear. The user can modify the logical relationship between the task and the execution plan by switching the text button. The agent will analyze

the task based on the user's instructions and return the results of multiple subtasks. During the exploration process, users can ask various questions and discuss them with LLM in the dialog box, such as the problem analysis method, understanding of the answer, and explanation for the recommendation.

Visualization view: This view presents visualizations and insights in the form of cards. Each card contains the task serial number, the task content, the interactive visualizations, and the insights in rich text format. Users can modify the Vega-Lite JSON codes and insight text. To address the issue of cognitive load for the user, we allow the user to select which visualization cards they want to view, and these selected cards are displayed in the main view while the remaining cards are moved to the candidate set below. Additionally, the user can merge the selected cards to create an interactive linked view. Moreover, users can modify and export the generated codes, which providing flexibility for those who need to customize their analyses further.

Task flow view: The task flow view updates as goals and tasks are added, providing the user with a clear analysis of status and progress (Fig. 4(C)). Each node is a rectangular box showing the task id, task type, and percent progress. The original task text appears when the cursor is over a node. Clicking on a node will update the visualization layout in the *Visualization view*. Hovering over the edge can see the differences in data and task type between the associated nodes. In accordance with design considerations, we allow users to choose unexplored tasks to be executed and delegate them to the agent. Additionally, users can remove pending tasks from the flow to reduce workload. A clear exploration structure may inspire user's ideas.

Data table view: In addition to task visualization, we provide a data lens visualization in *Data table view* (Fig. 4(D)) to guide users in the large exploration space. When a task is being completed, the user can observe the exploration of the accumulated data usage frequency in table lens mode. The color of the table cell represents the relative frequency of each cell being explored. This observation may inspire the user to discover regions of interest to propose new questions.

C. Error Handling

We refer to *errors* as issues that cause the system to crash or become unresponsive. Due to the inherent unpredictability of LLM outputs, errors may occur if the outputs cannot be parsed correctly, causing the system to crash or become unresponsive. To ensure the system operates correctly and prevents workflow disruptions, we implement an error-handling mechanism.

To develop an effective error-handling mechanism, we conducted a test using two datasets in expert evaluation with 20 agent-opposed tasks, employing both GPT-3.5-turbo and GPT-4-turbo. Each task was tested with code generation and insight annotation, utilizing two different prompting techniques. This resulted in a total of 160 initial tests. We classified the errors in the test into five categories: 1) *Unfamiliar dataset*, 2) *Data binding issues*, 3) *Serialization issues*, 4) *Data transformation issues*, 5) *Syntax errors*. More details about the test can be found

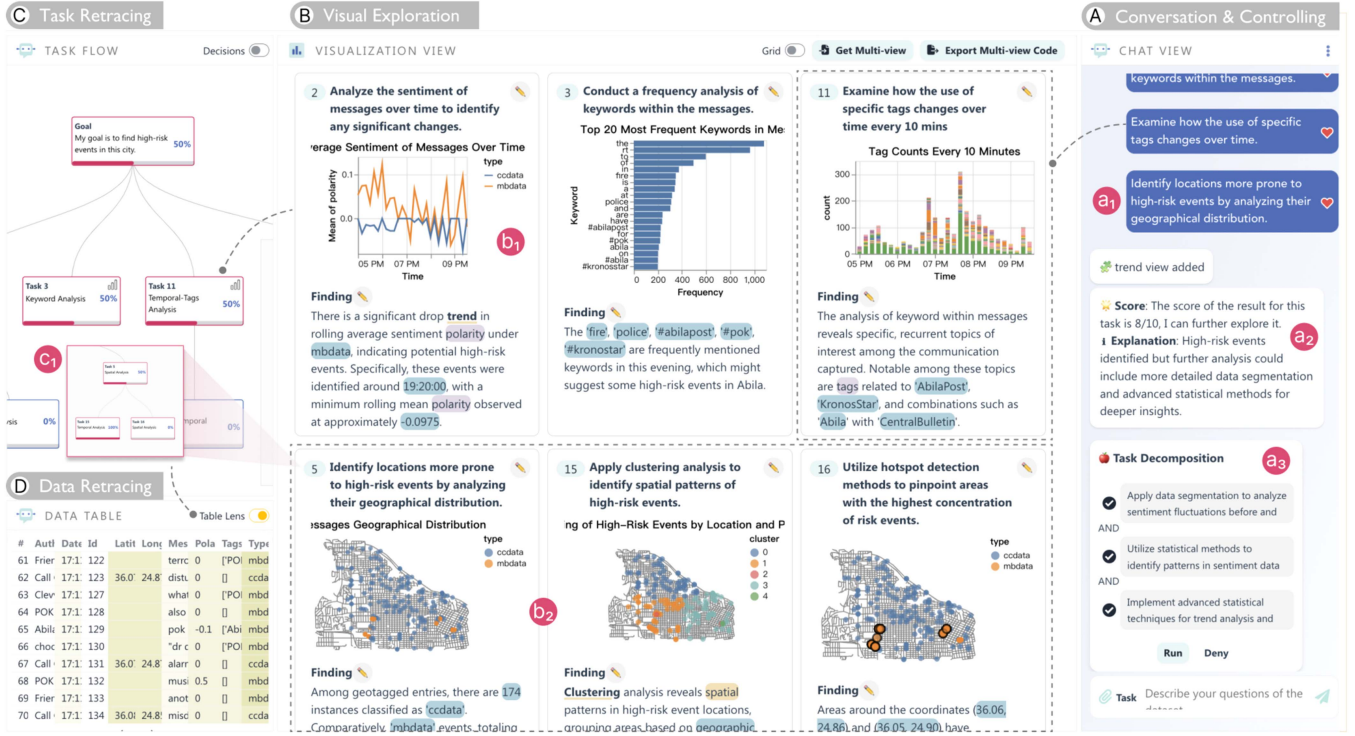


Fig. 4. The LightVA system comprises four views. Users can communicate with LLMs and control the planning process in (A) Chat view by selecting the tasks or setting the decomposition plan. The generated visualization and insights from LLMs are updated in (B) Visualization view. Task flow view (C) visualizes the task planning structure and allows users to control the analysis process. When a task is completed, users check the data exploration situation in the Data table with table lens (D).

in Appendix B, available online. To address these types of errors effectively, we implemented specific error-handling strategies in three ways: before model generation with prompting techniques, within the system after generation, or delegated to users.

Prompting techniques: 1) Few-shot prompting [61]: We provide examples to assist the model in better grasping the requirements of the outputs. For example, when making insight annotations, examples can explain the insight components of the model. 2) Chain-of-thoughts [52]: We guide the model through a thought plan with step-by-step instructions, which is helpful in reasoning processes, such as task execution.

Within system handling: 1) Self-reflection [62]: We allow LLMs to examine and correct their actions and outputs. For example, the model should debug codes based on the observation of the error. Based on our test results, the self-correction helped reduce around 40% initially identified errors, such as syntax errors, spelling mistakes, and logical inconsistencies. 2) Catching and feedback: Common syntax errors such as matching quotes and parentheses, can be solved by rule-based solutions. We classify these common errors to highlight which steps in the data analysis process the LLM made mistakes rather than merely pointing out low-level errors.

User-side handling: Once errors are identified, the system notifies the user, and the user may edit the codes. Additionally, we maintain the analysis history, allowing for a rollback to the previous step if an error occurs during the execution of the current task. Users can choose a new task and remove the failed task from the task flow.

V. USAGE SCENARIO: EVENT ANALYSIS

To examine the effectiveness of our framework, we demonstrate the LightVA with IEEE VAST Challenge 2021 Mini-Challenge 3² as a usage scenario. The challenge's goal is to detect events that happened in Abila City during the evening of January 23, 2014. The provided data include microblog records and emergency dispatch records from a call center. Our motivation for this scenario stems from two reasons. First, it incorporates a representative blend of data types and corresponding visualizations, encompassing text, spatial, and temporal data, which is a typical VA scenario. Second, the VAST challenge has the ground truth to support an objective evaluation of our LightVA. We use the OpenAI GPT-4 model in our work. For a more detailed demonstration of this scenario, a video is added to the supplemental materials.

Initialization: In the beginning, we upload a dataset.csv file and a map outline data.json file at the Chat view. Then, we type a goal, "find some high-risk events in this city". The dataset is introduced briefly, and the system proposes four initial exploration tasks based on our goal: *sentiment analysis*, *keyword analysis*, *tags analysis*, and *spatial analysis* (Fig. 4(a1)). As the four tasks are aligned with our objectives, we bookmark them all in our progress. We then prioritize the exploration of the first task - *sentiment analysis*. Upon submission, the agent generates a time varying card at the Visualization view

²<https://vast-challenge.github.io/2021/MC3.html>

(Fig. 4(B)). The generated insight for sentiment analysis indicates that the microblog records has experienced a drop in average sentiment, especially with a minimum polarity in 19:20, indicating potentially dangerous events (Fig. 4(b1)).

Task recommendation: The agent not only generates the visualization and findings but also evaluates them. For this *sentiment analysis* task, the agent gives a score of 8/10 and explains that the generated results have basically completed the task, but further statistical analysis can be performed (Fig. 4(a3)). As we prefer to start with a broad overview, we decide not to decompose. The agent thus reminds us to revisit our previous collection tasks in time. Thus, we choose the second task (*keyword analysis*), which involves generating a preliminary keyword visualization. The generated finding summarizes the highly frequent words such as “fire”, “police”, and “POK”. In addition to the agent’s finding, clicking over the stacked histogram for task 11 shows the time range of each word. For example, the “shooting” from 19:40 until 19:50 and “abirlafire” from 19:00 until 21:30. These findings suggest high-risk incidents like shootings and fires at first glance. The agent then recommends *spatial analysis*. From the *Data table* with lens, we can notice that there is indeed no latitude and longitude explored so far, which indicates that agent recommendation can pay attention to the data coverage (Fig. 4(D)). After execution, the system generates a map with shapes and messages, and the decomposition structure is updated on the Task flow (Fig. 4(c1)). We observe a higher concentration of microblogs in two specific locations, which might indicate incidents affecting public safety in those areas (Fig. 4(b2)).

Task decomposition: Based on the assessment of tasks and preliminary results, the agent recommends further decomposition with three subtasks: *clustering analysis*, *hotspots detection*, and *prediction modeling*, with an AND and DOWN logic (Fig. 4(a3)). To conduct a retrospective analysis, we choose the first two subtasks. The cluster analysis combines location and polarity to divide messages on the map into five categories. We find that two places on the left and right sides of the map have a distinct sentiment distribution. The second sub-task, through analyzing message quantity, highlights hotspots, visually indicating areas with dense messages.

Linked-view analysis: Based on the suggestion of the agent, we can form the analytical logic from multi-variables to concrete details. To find events more clearly, we select three tasks from the task flow view. The tasks include sentiment evolution charts, tag evolution charts, and a map to form an interactive multi-view (Fig. 5). Through interaction on line chart (Fig. 5(a)), we observe the histogram and map and discover that around 19:20, a “hit-and-run” event occurs: a “black van” first collides with a small car and then a cyclist, sparking a lot of discussions (Fig. 5(b) and (c)). Additionally, we find that the fire occurred near the hit-and-run site. Later, we find that the black van heads west by 19:44, and a shooting occurs with the police, marking the period with the highest discussion intensity. Finally, at 21:17, the van guys surrender. By observing the table and task flow, we find that the progress of the goal reached 100%, and the data is basically covered, “type” “latitude” and “longitude” were explored frequently.

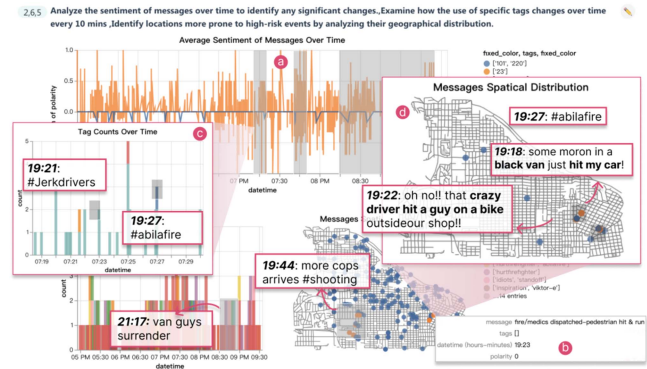


Fig. 5. A linked view for event analysis. The timeline, histograms, and map are selected by the user to generate a linked view. The charts can be brushed or clicked to filter each other (a). The polarity, tags, and message can be found in the tooltip (b) of each view to support detailed analysis. By interactively exploring the linked views (c, d), we find some dangerous events, such as “Hit and run”, “Standoff”, “Fire”.

To conclude, in this scenario, we find significant events such as Fire, Hit and Run, and Stand-off without manually coding and designing. With task planning, we generate eight views with auto-summarized findings based on statistical analysis and a linked view to solve the goal rapidly. Compared to our submission to the VAST Challenge [63], where we spent 2 people, 2 days, and 6 hours on data preprocessing (including tagging, categorizing, and removing spam messages) plus 2 people, 7 days, and 6 hours on interface construction and analysis, totaling around 108 hours. In LightVA, we upload the preprocessed data. Further construction and analysis take roughly 1 hour. This represents an efficiency increase of approximately 5 times, as the current effort is 25 out of 108 hours. Moreover, it is worth noting that data wrangling still require human involvement, especially in complex scenarios. One potential avenue would be to integrate agent-assisted data annotation and cleaning into LightVA’s workflow. In addition, there may be a lack of finesse and aesthetic appeal in interface visualizations and interactions when compared to manual designs. Despite this, the overall cost-effectiveness has been enhanced. To address this concern, we offer support for exporting the code, which can be further modified as needed.

VI. EXPERT STUDY

To examine the effectiveness of LightVA in VA system construction and task recommendation, we invited two VA experts (denoted as E1 and E2) and a domain expert (denoted as E3) to participate in the study. These experts have experience in data analysis and are familiar with using VA systems.

Participants background: E1 is a VA expert specializing in VA system development, topics around digital humanities, text-based data, road data for autonomous driving, spatiotemporal datasets, and tabular data. E1 often uses D3.js and Vue.js to develop VA systems. E2 is a VA expert with experience in autonomous driving and social media, as well as work experience in business analysis. In daily work, E2 chooses to use tools like Tableau within the company to analyze quantitative data. The

frequency of using data analysis and visualization tools is about weekly. E3 is a domain expert analyzing data for fast-moving consumer goods and supply chains. E3 frequently uses Excel and Power BI and occasionally uses Python for in-depth analysis. The use of data analysis and visualization tools occurs on a daily basis.

Procedure: The study consists of three sessions. First, we spent 15 minutes to know the experts' backgrounds in data analysis and introduced our work by showing the video demonstration. Then, in the exploration phase, the experts spend about 30 minutes exploring the system using the think-aloud method [64]. Considering E3's daily work requirements and aligning with their domain expertise, we opted for a dataset that mirrors their routine tasks. Due to data confidentiality concerns, we substituted the original dataset with a comparable one related to sales for E3's use. Meanwhile, the automotive dataset was designated for exploration by E1 and E2, fitting their respective areas of expertise. During the system usage stage, we observed and recorded how they interacted with the LightVA system. In the final stage of the interview, we spent approximately 30 minutes gathering the experts' feedback on their usage experience of the system.

A. Visual Analytics Expert Evaluation: Cars Dataset

In this study, we evaluate LightVA with E1 and E2 using a well-known Auto MPG dataset.³ The dataset has 406 records and 9 attributes, including brand, model, performance indicators (such as horsepower and cylinders), and the year of manufacture and place of origin.

During the exploration process, E1 is more concerned with the construction of the VA system. At the beginning, E1 sets the analytical goal to *identify the factors influencing fuel efficiency*. E1 chooses one of the initially proposed tasks for execution. After completing that task, the agent presents a detailed decomposition plan. E1 then follows the agent's guidance to continue selecting subsequent sub-tasks for further decomposition and in-depth analysis. In the generated visualization (Fig. 6(a1)), E1 discovers the differences in the distribution of the number of vehicles produced by different origins across various performance ranges in the bar chart on the right, by utilizing the area brushing feature on the scatter plot. E1 also discovers the performance differences between cars with different numbers of cylinders through the filter functionality between linked view (Fig. 6(a2)).

Another expert E2, demonstrates a focused interest in the data analysis, pursuing a clear analytical goal that was refined throughout the analysis process. Initially, E2's goal was *to identify vehicles that excel both in fuel efficiency and performance* (Fig. 6(b1)). However, recommended insights revealed a strong negative correlation between fuel efficiency (measured in MPG) and the majority of performance metrics, indicating a trade-off between high fuel efficiency and superior performance capabilities. This insight led E2 to quickly adjust the analytical goal towards *prioritizing fuel efficiency while ensuring performance was not significantly compromised* (Fig. 6(b2)). Based



Fig. 6. The illustrations of the expert study using our framework with cars dataset. Two VA experts show different analysis preferences during the exploration process. (A) E1 focuses on the creation of VA, progressively decomposing the goal and employing statistical methods to complete the exploration. (B) E2 has a clear goal during the analysis, and continuously optimizes the goal with task decomposition, and leverages the linked-view interactions to find a satisfactory answer.

on the plan proposed in task decomposition, E2 explored the relationship between various vehicle performance metrics and fuel efficiency with five charts. Finally, E2 combined these charts to generate linked view (Fig. 6(b3)). By filtering points across views, E2 found several vehicle models meeting the revised goal.

B. Domain Expert Evaluation: Sales Dataset

To further evaluate LightVA, we conducted another study with a domain expert. E3 was interested in a sales dataset for a large store that provides detailed transaction information.⁴ The dataset contains 3,312 records with 21 fields, e.g., sales, discount, profit, and category.

At the beginning, E3 mentioned the high-time sensitivity of sales data for products in real-world scenarios. Therefore, in the initial tasks, E3 opted to explore the temporal trends of product sales volumes. In the histogram and insight (Fig. 7(a1)) suggested by the agent, E3 found that the overall sales of the item differ significantly from month to month. In the next step of the decomposition plan, E3 categorized the products to explore the temporal trends of different categories (Fig. 7(a2)). It was

³<http://archive.ics.uci.edu/ml/datasets/Auto+MPG>

⁴<https://www.kaggle.com/datasets/addhyay/superstore-dataset>

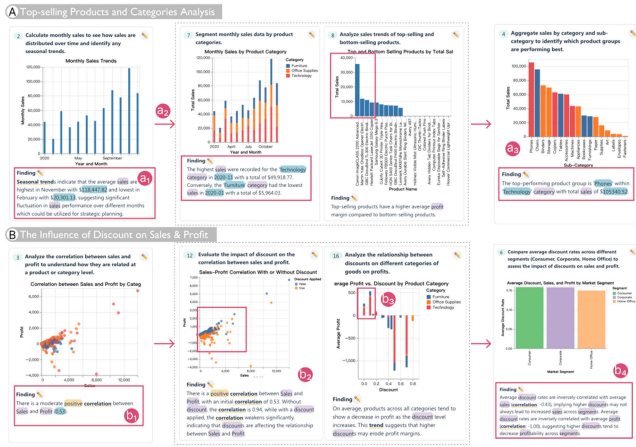


Fig. 7. The example results of the domain expert study with a superstore sales dataset. E3 implements analysis from overview to details and beyond with two rounds of task decomposition. (A) The top-selling products and specific categories are found for inventory formulation. (B) The system effectively pointed out the impact of discounts on sales and profits, promoting hypothesis generation and verification.

found that the temporal trends of the different categories were broadly similar, but the total sales volume of the technology category in November was the highest. Moreover, following the agent's decomposition into another task: "Analyze sales trends of top-selling and bottom-selling products", E3 noted that in the domain scenario, "finding out how well products are selling can optimize inventory to prevent stockouts and excess inventory." Therefore, E3 selected a task previously proposed but not executed, recommended for review by the agent, to rank the sales volumes of different categories and subcategories (Fig. 7(a3)). Based on the generated visualization, E3 discovered that the "phone" subcategory within the "Technology" category has the best sales performance.

After finding the answer to the previous task, E3 was interested in "analyze the correlation between sales and profit." Upon receiving the specific scatter plot (Fig. 7(b1)), E3 discovered that high sales do not always mean higher profit. The expert speculated that this may be due to the impact of discounts, as items with high sales often have significant discounts. E3 confirmed this hypothesis through interaction with and analysis of the scatter plots of sales and profit, with and without discount (Fig. 7(b2)). To find the optimal discount rate for products, E3 chose to analyze the relationship between discount and profit across different product categories. According to Fig. 7(b3), the differences between product categories are minor, with all reaching maximum profit at a 10% discount.

C. Results and Analysis

This section discusses the expert's exploration process and feedback on the system construction and task planning.

Exploration process analysis: The exploration behaviors of the three experts differ significantly. VA expert E1 created the most visualization charts, with half including interactive features, emphasizing the generation of visualizations and the exploratory use of linked views. Furthermore, E1 delved deeper

into task decomposition to employ more complex statistical methods, such as linear regression and polynomial regression. E2, with profound data analysis experience, did not strictly follow the agent's recommendations during the exploration process. Instead, E2 refined and improved the analysis goals and exploration directions based on the results of previous tasks, proposing new questions and flexibly employing interactive VA to complete the goal. E3, as a domain expert, possessed an understanding of the characteristics of certain data attributes within the dataset, such as the relationships between discounts, sales, and profit. E3's analysis was grounded in real-world scenarios, selecting different analysis methods as needed to integrate data insights into practical applications. This suggests our system supports a degree of flexibility, responding effectively to users' individual needs.

Feedback on system construction: E2 and E3 both expressed that interactive analysis through linked views facilitates a better understanding of the data. From the perspective of a VA expert, E1 commented that "basic charts and interactions can be generated, which are sufficient for simple data analysis problems, but modifications might be necessary for more complex issues." E2 and E3 agreed that the system can effectively generate insights, enhancing the accuracy of analysis. E2 suggested that generating insights based on awaring of user behavior would enhance the execution of tasks. Additionally, E3 noted the significant reduction in manual effort due to AI generation, stating, "Previously, using PowerBI required a lot of time and operations, but now it only takes a sentence."

Feedback on task planning: E3 observed that "the proposed tasks are quite good, indicating the system has a certain understanding of the dataset, and the language used is very standard." E1 valued the system's ability to decompose tasks as "the most useful part," which can provide deeper analysis and reveal certain characteristics of the data, especially for those lacking domain-specific expertise. However, domain expert E3 cautioned that the decomposed tasks are not always reliable, stating, "The process of task decomposition needs to be explained." Regarding task recommendations, E1 seeks more targeted suggestions that adhere closely to instructions without becoming too divergent. In contrast, E3 emphasizes that approaching goals from different perspectives can yield valuable insights and strategies in their work scenarios. These differing viewpoints highlight the need for the planning algorithm to be adaptable to user preferences.

VII. DISCUSSION

This section outlines the limitations of our work and discuss the implications learned from the research with the future directions.

Generalizability of the framework: Our framework is generalizable in two aspects. First, our conceptual framework unifies the development and usage of VA systems, based on the connections between goals, tasks, visualizations, and insights. Second, compared with existing visualization software, e.g., Tableau (with AI

version, Ask Data),⁵ LightVA offers several distinct advantages. While Tableau supports natural language interfaces, task recommendation, and single visualization generation, it lacks support for task decomposition, multi-view visualization generation, and iterative human-in-the-loop task completion evaluation, which are the focuses of LightVA.

Meanwhile, there are some limitations to our framework. First, the framework may require more detailed guidelines for complex and specific domains, particularly in task customization, visualization methods, and model selection. A more comprehensive grammar for tasks and insights would improve model output evaluation and refinement. This necessitates extensive research and documentation, categorizing different tasks and domains to enhance the framework's applicability. Second, functionality in the development of VA systems needs to be improved, particularly features that facilitate authoring and version iteration. Third, the relatively small number of participants in the expert study limits the generalizability of the results. Conducting a larger crowdsourced user study would be beneficial in verifying the quality of the automated output further and investigating the factors that contribute to users' perceived quality.

The performance of LLMs in VA tasks: While LLM exhibits potentials, some of the limitations of LLM requires careful handling.

- *Output stability and accuracy:* The output of LLMs can be unstable. During our evaluation, we encountered instances where the model failed to follow instructions accurately, leading to parsing and execution failures in the generated code. In order to address this problem, we propose an error-handling mechanism. Although this mechanism helps to avoid errors making system crashes, additional LLM errors about incorrect facts should be detected and corrected. In the future, we can incorporate LLMs' self-reflection to solve hallucination [65] and provide insights on errors so that users can work together to fix these errors.
- *Response speed:* From our test, GPT-3.5-turbo completes tasks in significantly less time than GPT-4-turbo. To increase the running speed, we use a multi-agent parallel computing strategy and error handling for time exceeding a certain threshold. A future direction could be to utilize caching mechanisms such as GPTCache [66] to explore acceleration strategies in data analysis scenarios.
- *Problem-solving ability:* Our test results indicate that LLMs can struggle with complex tasks, such as predictive modeling. This highlights the need for a broader evaluation of model capabilities across various complex domain-specific problems to derive guidelines or evaluation metrics for task planning and execution. Additionally, we found that LLMs may struggle with processing issues like missing values due to unfamiliarity with data. From the implementation side, we should provide additional materials or tools and teach agents to process data effectively [67].

- *Domain knowledge:* In some scenarios, LLMs may not have sufficient domain knowledge. To address this limitation, future research can focus on combining retrieval augmented generation (RAG) and fine-tuning for LLMs to solve specific domain tasks [68].

Injecting visualization design knowledge: Design knowledge is important in designing VA systems. In our implementation, we constrain color, interaction, and layout. These constraints are not exhaustive and we only consider them as preliminary. Further research can include more design guidelines for the LLM with prompting techniques or multimodal models [69] to perceive and evaluate the effectiveness. Besides, a memory module [70] can be integrated to make the agent evolve. In addition, interactions other than natural language can be designed to facilitate user input of intentions, such as sketching [71] or generating widgets [72].

Automation and personalization: From the user study, we found that users from different backgrounds have different needs. Some users have a clear idea of what they want and need the agent to follow orders in detail, while others prefer the agent to take the lead in a more automated way. This divergence in user preferences highlights the balance between user agency (the control they maintain over decisions) and the automation of processes. In future studies, we could explore how different levels of agency and automation affect task performance and user satisfaction [32]. Additionally, users may require different levels of assistance in terms of breadth and depth. We could conduct further qualitative experiments to test the differences in decision-making paths when experts in visualization are assisted by agents [73], [74], comparing these paths visually using graph algorithms. This could help us build a preference knowledge base for different user types, enabling the model to simulate and adapt to users' desired paths, providing more personalized recommendations.

VIII. CONCLUSION

In this paper, we aim to reduce the complexities and technical demands of carrying out visual analytics. Therefore, We introduce LightVA, a lightweight visual analytics framework that supports task planning, insight analysis, and linked visualization generation based on human-agent collaboration. Our framework utilizes LLM agents for task planning and execution. The framework employs a recursive approach in which the agents recommend tasks, break down complex tasks into subtasks, and generate visualization and data modeling codes to solve tasks. We develop a system that embodies our proposed framework, supporting users to analyze data based on the communication with LLM agents and use the task-driven generated VA system. A usage scenario and an expert study suggest that LightVA not only reduced the manual effort required but also provided new opportunities to leverage LLMs to facilitate visual data exploration.

ACKNOWLEDGMENT

The authors thank anonymous reviewers for their constructive comments.

⁵https://help.tableau.com/current/pro/desktop/en-us/ask_data.htm, last accessed March 2024

REFERENCES

- [1] J. J. Thomas and K. A. Cook, *Illuminating the Path: An R&D Agenda for Visual Analytics*. National Visualization and Analytics Ctr, Richland, Washington, USA, 2005, pp. 69–104.
- [2] D. Keim, G. Andrienko, J.-D. Fekete, C. Görg, J. Kohlhammer, and G. Melançon, “Visual analytics: Definition, process, and challenges,” in *Information Visualization: Human-Centered Issues and Perspectives*, Berlin, Germany: Springer, 2008, pp. 154–175.
- [3] A. Wu, D. Deng, F. Cheng, Y. Wu, S. Liu, and H. Qu, “In defence of visual analytics systems: Replies to critics,” *IEEE Trans. Vis. Comput. Graphics*, vol. 29, no. 1, pp. 1026–1036, Jan. 2023.
- [4] D. Ceneda et al., “Characterizing guidance in visual analytics,” *IEEE Trans. Vis. Comput. Graphics*, vol. 23, no. 1, pp. 111–120, Jan. 2017.
- [5] D. Deng, A. Wu, H. Qu, and Y. Wu, “DashBot: Insight-driven dashboard generation based on deep reinforcement learning,” *IEEE Trans. Vis. Comput. Graphics*, vol. 29, no. 1, pp. 690–700, Jan. 2023.
- [6] B. Yu and C. T. Silva, “FlowSense: A natural language interface for visual data exploration within a dataflow system,” *IEEE Trans. Vis. Comput. Graphics*, vol. 26, no. 1, pp. 1–11, Jan. 2020.
- [7] Y. Wang et al., “DataShot: Automatic generation of fact sheets from tabular data,” *IEEE Trans. Vis. Comput. Graphics*, vol. 26, no. 1, pp. 895–905, Jan. 2020.
- [8] C. Demiralp, P. J. Haas, S. Parthasarathy, and T. Pedapati, “Foresight: Recommending visual insights,” *Proc. VLDB Endowment*, vol. 10, no. 12, pp. 1937–1940, 2017.
- [9] S. Hao et al., “Reasoning with language model is planning with world model,” in *Proc. Conf. Empir. Methods Natural Lang. Process.*, H. Bouamor, J. Pino, and K. Bali, Eds., 2023, pp. 8154–8173.
- [10] S. Sharan et al., “LLM-Assist: Enhancing closed-loop planning with language-based reasoning,” 2023, *arXiv:2401.00125*.
- [11] J. Yang, A. Prabhakar, K. Narasimhan, and S. Yao, “InterCode: Standardizing and benchmarking interactive coding with execution feedback,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 23826–23854.
- [12] S.-C. Liu et al., “JarviX: A LLM no code platform for tabular data analysis and optimization,” in *Proc. Conf. Empir. Methods Natural Lang. Process.: Ind. Track*, 2023, pp. 622–630.
- [13] V. Dibia, “LIDA: A tool for automatic generation of grammar-agnostic visualizations and infographics using large language models,” in *Proc. 61st Annu. Meeting Assoc. Comput. Linguistics*, D. Bollegala, R. Huang, and A. Ritter, Eds., 2023, pp. 113–126.
- [14] L. Wang, S. Zhang, Y. Wang, E.-P. Lim, and Y. Wang, “LLM4Vis: Explainable visualization recommendation using ChatGPT,” 2023, *arXiv:2310.07652*.
- [15] R. Mao, G. Chen, X. Zhang, F. Guerin, and E. Cambria, “GPTEval: A survey on assessments of ChatGPT and GPT-4,” 2023, *arXiv:2308.12488*.
- [16] W. Zhang, Y. Shen, W. Lu, and Y. Zhuang, “Data-Copilot: Bridging billions of data and humans with autonomous workflow,” 2023, *arXiv:2306.07209*.
- [17] Y. Zhao et al., “LEVA: Using large language models to enhance visual analytics,” *IEEE Trans. Vis. Comput. Graphics*, early access, Mar. 4, 2024, doi: [10.1109/TVCG.2024.3368060](https://doi.org/10.1109/TVCG.2024.3368060).
- [18] A. Wu et al., “AI4VIS: Survey on artificial intelligence approaches for data visualization,” *IEEE Trans. Vis. Comput. Graphics*, vol. 28, no. 12, pp. 5049–5070, Dec. 2022.
- [19] Q. Wang, Z. Chen, Y. Wang, and H. Qu, “A survey on ML4VIS: Applying machine learning advances to data visualization,” *IEEE Trans. Vis. Comput. Graphics*, vol. 28, no. 12, pp. 5134–5153, Dec. 2022.
- [20] P. Ren, Y. Wang, and F. Zhao, “Re-understanding of data storytelling tools from a narrative perspective,” *Vis. Intell.*, vol. 1, no. 1, 2023, Art. no. 11.
- [21] K. Wongsuphasawat, D. Moritz, A. Anand, J. Mackinlay, B. Howe, and J. Heer, “Voyager: Exploratory analysis via faceted browsing of visualization recommendations,” *IEEE Trans. Vis. Comput. Graphics*, vol. 22, no. 1, pp. 649–658, Jan. 2016.
- [22] K. Wongsuphasawat, D. Moritz, A. Anand, J. Mackinlay, B. Howe, and J. Heer, “Towards a general-purpose query language for visualization recommendation,” in *Proc. Workshop Hum.-In-the-Loop Data Analytics*, 2016, pp. 1–6.
- [23] V. Dibia and Ç. Demiralp, “Data2Vis: Automatic generation of data visualizations using sequence to sequence recurrent neural networks,” *IEEE Comput. Graph. Appl.*, vol. 39, no. 5, pp. 33–46, Sep./Oct. 2019.
- [24] K. Hu, M. A. Bakker, S. Li, T. Kraska, and C. Hidalgo, “VizML: A machine learning approach to visualization recommendation,” in *Proc. SIGCHI Conf. Hum. Factors Comput. Syst.*, 2019, pp. 1–12.
- [25] M. Zhou et al., “Table2Charts: Recommending charts by learning shared table representations,” in *Proc. ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2021, pp. 2389–2399.
- [26] J. Zhao, M. Fan, and M. Feng, “ChartSeer: Interactive steering exploratory visual analysis with machine intelligence,” *IEEE Trans. Vis. Comput. Graphics*, vol. 28, no. 3, pp. 1500–1513, Mar. 2022.
- [27] S. Zhang, Y. Wang, H. Li, and H. Qu, “AdaVis: Adaptive and explainable visualization recommendation for tabular data,” *IEEE Trans. Vis. Comput. Graphics*, vol. 30, no. 9, pp. 5923–5938, Sep. 2024.
- [28] H. Li, Y. Wang, S. Zhang, Y. Song, and H. Qu, “KG4Vis: A knowledge graph-based approach for visualization recommendation,” *IEEE Trans. Vis. Comput. Graphics*, vol. 28, no. 1, pp. 195–205, Jan. 2022.
- [29] D. Raghunandan et al., “Lodestar: Supporting independent learning and rapid experimentation through data-driven analysis recommendations,” 2022, *arXiv:2204.07876*.
- [30] Z. Qu and J. Hullman, “Keeping multiple views consistent: Constraints, validations, and exceptions in visualization authoring,” *IEEE Trans. Vis. Comput. Graphics*, vol. 24, no. 1, pp. 468–477, Jan. 2018.
- [31] M. Sun, A. Namburi, D. Koop, J. Zhao, T. Li, and H. Chung, “Towards systematic design considerations for visualizing cross-view data relationships,” *IEEE Trans. Vis. Comput. Graphics*, vol. 28, no. 12, pp. 4741–4756, Dec. 2022.
- [32] H. Lin, D. Moritz, and J. Heer, “Dziban: Balancing agency & automation in visualization design via anchored recommendations,” in *Proc. SIGCHI Conf. Hum. Factors Comput. Syst.*, 2020, pp. 1–12.
- [33] D. Moritz et al., “Formalizing visualization design knowledge as constraints: Actionable and extensible models in draco,” *IEEE Trans. Vis. Comput. Graphics*, vol. 25, no. 1, pp. 438–448, Jan. 2019.
- [34] Y. Lin, H. Li, A. Wu, Y. Wang, and H. Qu, “DMiner: Dashboard design mining and recommendation,” *IEEE Trans. Vis. Comput. Graphics*, vol. 30, no. 7, pp. 4108–4121, Jul. 2024.
- [35] A. Wu et al., “MultiVision: Designing analytical dashboards with deep learning based recommendation,” *IEEE Trans. Vis. Comput. Graphics*, vol. 28, no. 1, pp. 162–172, Jan. 2022.
- [36] D. Shi, W. Cui, D. Huang, H. Zhang, and N. Cao, “Reverse-engineering information presentations: Recovering hierarchical grouping from layouts of visual elements,” *Vis. Intell.*, vol. 1, no. 1, 2023, Art. no. 9.
- [37] X. Li, Y. Zhang, J. Leung, C. Sun, and J. Zhao, “EDAssistant: Supporting exploratory data analysis in computational notebooks with in situ code search and recommendation,” *ACM Trans. Interactive Intell. Syst.*, vol. 13, no. 1, pp. 1–27, 2023.
- [38] O. Bar El, T. Milo, and A. Somech, “Automatically generating data exploration sessions using deep reinforcement learning,” in *Proc. 2020 ACM SIGMOD Int. Conf. Manage. Data*, 2020, pp. 1527–1537.
- [39] S. M. Casner, “Task-analytic approach to the automated design of graphic presentations,” *ACM Trans. Graph.*, vol. 10, no. 2, pp. 111–151, 1991.
- [40] B. Saket, A. Ender, and Ç. Demiralp, “Task-based effectiveness of basic visualizations,” *IEEE Trans. Vis. Comput. Graphics*, vol. 25, no. 7, pp. 2505–2512, Jul. 2019.
- [41] R. Amar, J. Eagan, and J. Stasko, “Low-level components of analytic activity in information visualization,” in *Proc. IEEE Symp. Inf. Visual.*, 2005, pp. 111–117.
- [42] D. Gotz and Z. Wen, “Behavior-driven visualization recommendation,” in *Proc. 14th Int. Conf. Intell. User Interfaces*, 2009, pp. 315–324.
- [43] F. Bouali, A. Guettala, and G. Venturini, “VizAssist: An interactive user assistant for visual data mining,” *Vis. Comput.*, vol. 32, pp. 1447–1463, 2016.
- [44] L. Shen, E. Shen, Z. Tai, Y. Song, and J. Wang, “TaskVis: Task-oriented visualization recommendation,” in *Proc. Eurographics Conf. Vis.*, 2021, pp. 91–95.
- [45] A. Pandey, A. Srinivasan, and V. Setlur, “MEDLEY: Intent-based recommendations to support dashboard composition,” *IEEE Trans. Vis. Comput. Graphics*, vol. 29, no. 1, pp. 1135–1145, Jan. 2023.
- [46] Y. Tian et al., “ChartGPT: Leveraging llms to generate charts from abstract natural language,” *IEEE Trans. Vis. Comput. Graphics*, early access, Feb. 22, 2024, doi: [10.1109/TVCG.2024.3368621](https://doi.org/10.1109/TVCG.2024.3368621).
- [47] G. Li et al., “Visualization generation with large language models: An evaluation,” 2024, *arXiv:2401.11255*.
- [48] Y. Huang et al., “Interactive table synthesis with natural language,” *IEEE Trans. Vis. Comput. Graphics*, vol. 30, no. 9, pp. 6130–6145, Sep. 2024.
- [49] M. M. Hassan, A. Knipper, and S. K. K. Santu, “ChatGPT as your personal data scientist,” 2023, *arXiv:2305.13657*.

- [50] P. Ma, R. Ding, S. Wang, S. Han, and D. Zhang, "Demonstration of InsightPilot: An LLM-empowered automated data exploration system," 2023, *arXiv:2304.00477*.
- [51] X. He et al., "Text2Analysis: A benchmark of table question answering with advanced data analysis and unclear queries," 2023, *arXiv:2312.13671*.
- [52] J. Wei et al., "Chain-of-Thought prompting elicits reasoning in large language models," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 24824–24837.
- [53] S. Yao et al., "Tree of Thoughts: Deliberate problem solving with large language models," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 11809–11822.
- [54] M. Besta et al., "Graph of thoughts: Solving elaborate problems with large language models," 2023, *arXiv:2308.09687*.
- [55] T. Wu, M. Terry, and C. J. Cai, "AI Chains: Transparent and controllable human-AI interaction by chaining large language model prompts," in *Proc. SIGCHI Conf. Hum. Factors Comput. Syst.*, 2022, pp. 1–22.
- [56] Y. Guo, D. Shi, M. Guo, Y. Wu, N. Cao, and Q. Chen, "Talk2Data: A natural language interface for exploratory visual analysis via question decomposition," *ACM Trans. Interactive Intell. Syst.*, vol. 14, 2024, Art. no. 8.
- [57] M. Q. Wang Baldonado, A. Woodruff, and A. Kuchinsky, "Guidelines for using multiple views in information visualization," in *Proc. Work. Conf. Adv. Vis. Interfaces*, 2000, pp. 110–119.
- [58] S. Khan, P. H. Nguyen, A. Abdul-Rahman, E. Freeman, C. Turkay, and M. Chen, "Rapid development of a data visualization service in an emergency response," *IEEE Trans. Serv. Comput.*, vol. 15, no. 3, pp. 1251–1264, May/Jun. 2022.
- [59] R. Ding, S. Han, Y. Xu, H. Zhang, and D. Zhang, "QuickInsights: Quick and automatic discovery of insights from multi-dimensional data," in *Proc. Int. Conf. Manage. Data*, 2019, pp. 317–332.
- [60] A. Satyanarayan, D. Moritz, K. Wongsuphasawat, and J. Heer, "Vega-Lite: A grammar of interactive graphics," *IEEE Trans. Vis. Comput. Graphics*, vol. 23, no. 1, pp. 341–350, Jan. 2017.
- [61] T. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., 2020, pp. 1877–1901.
- [62] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Proc. 37th Int. Conf. Neural Inf. Process. Syst.*, 2024, pp. 8634–8652.
- [63] L. Peng et al., "Mixed-initiative visual exploration of social media text and events," in *Proc. IEEE Conf. Visual. Vis. Analytics*, 2021. [Online]. Available: https://virtual.ieeevis.org/year/2021/poster_avastchallenge-posters-1019.html
- [64] M. Van Someren, Y. F. Barnard, and J. Sandberg, *The Think Aloud Method: A Practical Approach to Modelling Cognitive*. London, U.K.: Academic Press, 1994.
- [65] Z. Ji, T. Yu, Y. Xu, N. Lee, E. Ishii, and P. Fung, "Towards mitigating LLM hallucination via self reflection," in *Proc. Findings Assoc. Comput. Linguistics*, 2023, pp. 1827–1843.
- [66] F. Bang, "GPTCache: An open-source semantic cache for LLM applications enabling faster answers and cost savings," in *Proc. 3rd Workshop Natural Lang. Process. Open Source Softw.*, 2023, pp. 212–218.
- [67] Y. Qin et al., "ToolLLM: Facilitating large language models to master 16000 real-world APIs," 2023, *arXiv:2307.16789*.
- [68] L. Gao et al., "Fine-tuned large language model for visualization system: A study on self-regulated learning in education," *IEEE Trans. Vis. Comput. Graphics*, vol. 31, no. 1, pp. 514–524, Jan. 2025.
- [69] X. Zeng, H. Lin, Y. Ye, and W. Zeng, "Advancing multimodal large language models in chart question answering with visualization-referenced instruction tuning," *IEEE Trans. Vis. Comput. Graphics*, vol. 31, no. 1, pp. 525–535, Jan. 2025.
- [70] C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," 2023, *arXiv:2310.08560*.
- [71] D. Masson, S. Malacria, G. Casiez, and D. Vogel, "DirectGPT: A direct manipulation interface to interact with large language models," in *Proc. 2024 CHI Conf. Hum. Factors Comput. Syst.*, 2024, Art. no. 975.
- [72] P. Vaithilingam, E. L. Glassman, J. P. Inala, and C. Wang, "DynaVis: Dynamically synthesized UI widgets for visualization editing," in *Proc. CHI Conf. Hum. Factors Comput. Syst.*, 2024, Art. no. 985.
- [73] L. Battle and J. Heer, "Characterizing exploratory visual analysis: A literature review and evaluation of analytic provenance in tableau," *Comput. Graph. Forum*, vol. 38, no. 3, pp. 145–159, 2019.
- [74] Y. Liu, T. Althoff, and J. Heer, "Paths explored, paths omitted, paths obscured: Decision points & selective reporting in end-to-end data analysis," in *Proc. SIGCHI Conf. Hum. Factors Comput. Syst.*, 2020, pp. 1–14.



Yuheng Zhao is currently working toward the PhD degree with the School of Data Science, Fudan University. Her primary research interests are visualization and visual analytics, with an emphasis on foundation model-powered intelligent visual analytics. For more information, please visit <https://zyh1222.github.io/>.



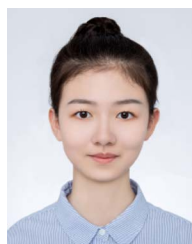
Junjie Wang received the BS degree from Fudan University. He is currently working toward the master's degree with the School of Data Science, Fudan University. His primary research interests are visualization and visual analytics, with a specific focus on intelligent and adaptive visual analytics.



Linbing Xiang received the BS degree in software engineering from Fudan University, she is currently working toward the master's degree in Big Data Technology with the Hong Kong University of Science and Technology. Her primary research interests focus on applied artificial intelligence, specifically emphasizing data visualization and visual analytics within the realm of generative AI.



Xiaowen Zhang is currently working toward the master's degree with the School of Data Science, Fudan University. Her primary research interests lie in visualization and visual analytics, with a particular focus on visual analytics in the field of fintech.



Zifei Guo is currently working toward the undergraduate degree with the School of Mathematical Sciences, Fudan University. Her primary research interests are data science, visual analytics, and machine learning application.



Cagatay Turkay is a professor with the Centre for Interdisciplinary Methodologies, University of Warwick, U.K., and a Turing fellow with Alan Turing Institute, London, U.K. His research investigates the interactions between data, algorithms and people, and explores the role of interactive visualisation and other interaction mediums such as natural language at this intersection. He frequently publishes his research on visualisation journals such as *IEEE Transactions on Visualization and Computer Graphics*, *Computer Graphics Forum*, and *IEEE Computer Graphics and Applications*, as well as journals in machine learning and data mining. He has been awarded the EuroVis Young Researcher 2019 award and named a EuroGraphics Junior fellow, in 2019.



Yu Zhang received the BS degree in Intelligence Science and Technology from Peking University, in 2017. He is currently working toward the PhD degree with the Department of Computer Science, University of Oxford. His research focuses on intelligent user interfaces in the field of human-computer interaction.



Siming Chen received the PhD degree in computer science from Peking University. He is an associate professor with the School of Data Science, Fudan University. Prior to this, he was a research scientist with Fraunhofer Institute IAIS in Germany. His research interests are visualization and visual analytics, with the emphasis on Human-AI Collaboration, including LLM-driven visual analytics, social media and autonomous driving visual analytics. He has published 100 papers and more than 40 in top conferences and journals, including *IEEE VIS*, *IEEE Transactions on Visualization and Computer Graphics*, *EuroVis*, *ACM CHI*, *UIST*, *CSCW*, etc. He was awarded more than 10 best paper/poster awards and honorable mentioned awards in multiple conferences. For more information, please visit <http://simingchen.me>.